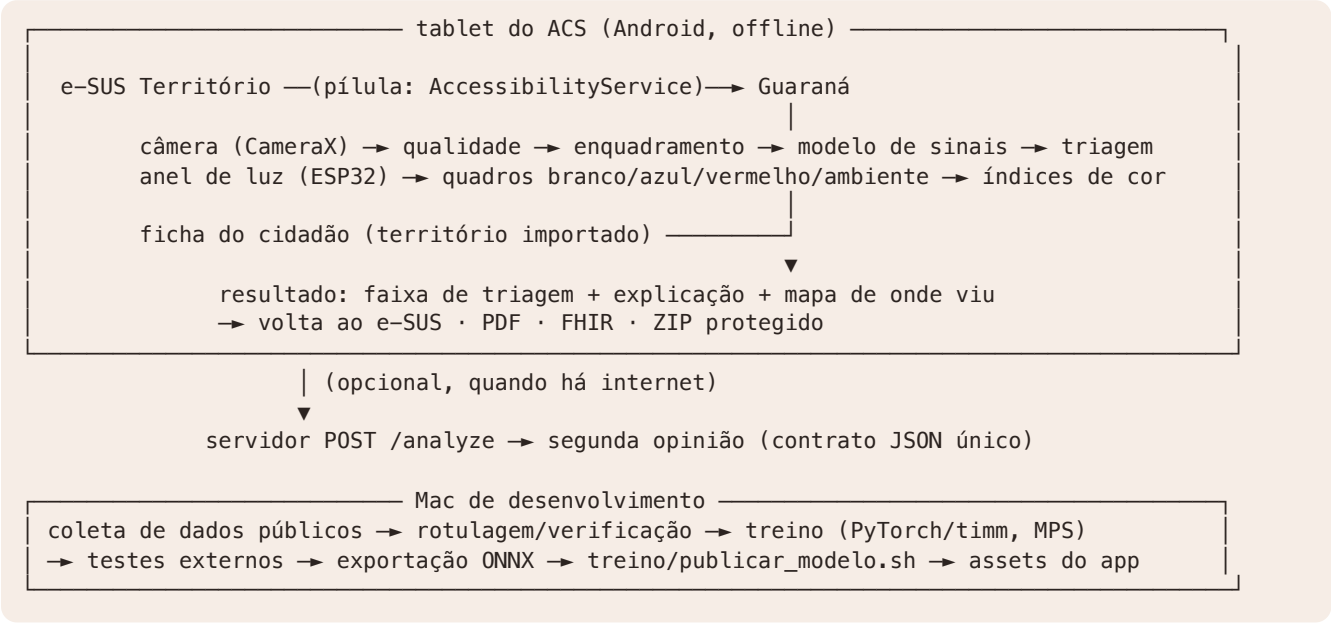


Guaraná • Guia técnico

Como o Guaraná funciona por dentro: o app Android, a integração com o e-SUS, o hardware de luz, os modelos, os dados e o treino. Para o porquê do produto, veja a [tese](#). Para o histórico de cada experimento, o [diário técnico](#).

Versão de 01/10/2026.

1. Visão geral



Três princípios atravessam o código:

1. **Offline primeiro.** Todo modelo roda no aparelho via ONNX Runtime. A nuvem só confirma.
2. **Um contrato.** O motor local, a nuvem e o app falam o mesmo JSON (`contrato/resultado.schema.json`).
3. **Régua honesta.** Nenhum número é reportado sem teste em dados de fonte que o modelo nunca viu.

2. Repositório

Pasta	Conteúdo
<code>guarana-app/</code>	app Android (Kotlin, Jetpack Compose)
<code>esus-mock-app/</code>	simulador do e-SUS Território (dados fictícios) para testar a integração
<code>firmware/</code>	anel de luz ESP32: serial (padrão), BLE/Wi-Fi e simulador HTTP
<code>treino/</code>	coleta, preparação, treino, avaliação e publicação de modelos (Python)
<code>synthese/</code>	olhos sintéticos no Blender (experimental)
<code>contrato/</code>	esquema JSON do resultado
<code>docs/</code>	tese, este guia e o diário técnico

Fora do git (gerados localmente): `dados/` (datasets de terceiros, ~17 GB), `saida/` (pesos, cartões, relatórios), `.venv/` , renders e builds.

3. App Android

3.1 Stack

Camada	Tecnologia
Linguagem e UI	Kotlin, Jetpack Compose (Material 3), minSdk 26 , targetSdk 35
Câmera	CameraX 1.4 (preview, captura, ImageAnalysis para dica ao vivo), interop Camera2 para travar exposição e foco
Inferência	ONNX Runtime Android 1.20 (CPU)
Mapa	MapLibre 11 (tiles baixáveis para uso offline)
Hardware	usb-serial-for-android (USB/OTG), BLE do Android, OkHttp (Wi-Fi)
Exportação	PDF nativo do Android, FHIR JSON, ZIP AES-256 (zip4j)
ABIs	arm64-v8a, armeabi-v7a, x86_64 (tablets 32 bits e emulador)

3.2 Pacotes

com.guarana.ocular	
└─ MainActivity.kt	navegação, entrada vinda do e-SUS, botão voltar que devolve ao e-SUS
└─ ui/	telas Compose
└─ HomeScreen	início: resumo do dia, visitas, nova análise
└─ AgendaScreen	visitas ordenadas por prioridade ocular
└─ TerritoryScreen	bairros > logradouros > imóveis > famílias (hierarquia do e-SUS)
└─ MapScreen/HouseholdMap	mapa dos domicílios (MapLibre)
└─ PatientsScreen	busca e histórico
└─ CaptureScreen	câmera, moldura, dica de enquadramento ao vivo, rotina de luz
└─ ProcessingScreen	progresso da análise
└─ ResultScreen	triagem, sinais, mapa de calor, explicação, e-SUS, exportação
└─ SettingsScreen	nuvem, território, anel de luz, validação, enquadramento
└─ Logo, Theme, Components	
└─ core/	lógica sem UI
└─ Pipeline	orquestra uma análise (ver 3.3)
└─ QualityGate	nitidez, exposição, reflexo
└─ EyeFraming	modelo de enquadramento
└─ EyeSegmenter	modelo de fenda e íris
└─ LocalAnalyzer	modelo de sinais + vistas + mapa de ativação
└─ ModelCard	lê o cartão do modelo (classes, limiares, contrato de entrada)
└─ Triage	regras de triagem pela foto e pela ficha
└─ OcularPriority	pontuação de prioridade ocular por pessoa
└─ Analysis, Patient, Territory, Store, Settings	
└─ RemoteAnalyzer	cliente da nuvem (POST /analyze)
└─ Report	PDF e ZIP protegido
└─ Validation	validação automática numa pasta de fotos
└─ color/Reflectance	ramo de cor (índices de absorbância)
└─ light/	LightRig: UsbSerial, Ble, Http
└─ fhir/FhirExport	Bundle FHIR para a RNDS
└─ esus/	
└─ EsusCompanionService	pílula flutuante sobre o e-SUS (AccessibilityService)
└─ Companion	ponte e-SUS ↔ Guaraná, resumo para a ficha de visita

3.3 Uma análise de ponta a ponta (`core/Pipeline.kt`)

```
fun run(file: File, child: Boolean, frames: List<Pair<String, File>>, patientId: String): Analysis
{
    var bitmap = Store.decodeOriented(file)
    var q0 = QualityGate.check(bitmap)
    // reflexo forte (>4% de pixels estourados) e quadro sem luz limpo: analisa o quadro sem luz
    if (q0.metrics["reflexo"] > 0.04f) { /* troca para o quadro "ambiente" */ }
    val framing = framingGate.check(bitmap)
    if (framing.rejects && settings.requireEyeFraming) throw FramingRejected(framing)
    var result = store.save(bitmap, local.analyze(bitmap, quality, child) ...) // modelo de
    sinais + triagem
    // ramo de cor: red-free por software e índices na esclera (ROI pelo segmentador)
    ReflectanceIndices.compute(frames, segmenter)?.let { ... result.copy(indices = refl.toMap()) }
    // nuvem opcional: confirma ou marca divergência
    if (settings.useCloud) RemoteAnalyzer(url).analyze(...)?.let { status = if (igual) "confirmed"
    else "divergent" }
    return result
}
```

3.4 Captura (`ui/CaptureScreen.kt`)

- **Moldura oval** e recorte igual ao preview (ViewPort), para o que se vê ser o que se analisa.
- **Dica ao vivo:** um `ImageAnalysis` subamostra o plano Y de cada quadro e roda o modelo de enquadramento (~ms). A pílula da dica muda de cor: "Aproxime-se do olho", "Um olho de cada vez", "Olho enquadrado".
- **Rotina de luz** (com o anel): branco, azul, vermelho e ambiente. Depois do primeiro quadro, exposição e balanço de branco ficam **travados** (Camera2 interop), para os quadros serem comparáveis e o ambiente poder ser subtraído.
- **Sem anel:** o flash do tablet pode fazer o papel do branco (`TORCH on/off` na rotina).
- **Galeria:** analisa uma foto existente (útil para testes).

3.5 Portões antes do modelo

Portão	Arquivo	Regra
Qualidade	<code>QualityGate.kt</code>	desfoque se variância do laplaciano < 30; escura se luminância média < 45; superexposta se > 215; reflexo se > 4% de pixels estourados
Enquadramento	<code>EyeFraming.kt</code> + <code>enquadramento_v1.onnx</code>	classes OK, TOO_FAR, TOO_CLOSE, TWO_EYES, OFF_CENTER, NOT_FOUND. Recusa só TOO_FAR, TWO_EYES e NOT_FOUND; descentralizado e perto demais viram aviso

3.6 Modelo de sinais (`core/LocalAnalyzer.kt`)

1. **Vistas:** recorte central, as duas laterais (ou topo e base) e o espelho de cada uma. A probabilidade de cada sinal é o **máximo entre as vistas**, para lesões na borda (pterígio) não se perderem no recorte central.
2. **Mapa de ativação:** o ONNX tem uma segunda saída `cam` (cabeça aplicada a cada célula da grade). O app desenha o mapa da vista vencedora sobre a foto e gera a frase "viu na parte branca do olho, à direita".
3. **Estados:** dois limiares por sinal, lidos do cartão do modelo.

```
val thr = card.thresholds[id] // "presente": 98,5% de especificidade em
olhos normais reais
val thrC = card.candidateThresholds[id] ?: thr / 2f // "candidato": 92%
val state = when { p >= thr -> present; p >= thrC && p >= 0.05f -> candidate; else -> absent }
```

1. **Confiabilidade:** sinais sem validação em foto comum (`unreliable_labels` no cartão) aparecem marcados como tal na tela.

3.7 Triage (core/Triage.kt e core/OcularPriority.kt)

Pela foto:

```
child && "catarata_leuocoria" in present -> "encaminhar_urgente" // retinoblastoma / catarata
congénita
present.any { it in URGENTE } -> "encaminhar_urgente" // ceratite, uveíte
present.any { it in ENCAMINHAR }-> "encaminhar" // pterígio, catarata, icterícia,
opacidade, lesão pigmentada, tumor...
present.any { it in OBSERVAR } -> "observar" // hiperemia, hemorragia, conjuntivite,
pálpebra...
signs.any { candidate } -> "observar"
else -> "sem_sinais"
```

Pela ficha do cidadão (withHistory), regras provisórias a validar com oftalmologista:

- diabetes sem fundo de olho registrado em 12 meses → pelo menos **encaminhar**, mesmo com foto normal;
- deficiência visual ou hanseníase + qualquer sinal → **encaminhar**;
- hipertensão + hemorragia subconjuntival → **observar** e aferir a pressão.

OcularPriority soma fatores da ficha (idade, diabetes, hipertensão, hanseníase, deficiência visual, tempo sem visita) e ordena a agenda (sortVisits): prioridade alta ≥ 4 pontos, média ≥ 2 .

3.8 Explicação para leigo (ui/ResultScreen.kt)

A primeira coisa na tela é linguagem simples ("Viu vermelhidão na parte branca do olho, à direita, no meio da foto. Muito provável (91%)") e a qualidade da foto em $\sqrt{!}$. Os números técnicos (variância do laplaciano, limiares, vistas) ficam atrás de um ícone de expandir.

3.9 Integração com o e-SUS (esus/)

O e-SUS Território não tem código aberto nem API para apps de terceiros. A integração é **pela tela**:

- EsusCompanionService é um AccessibilityService. Ele observa qual app está ativo e, quando é o e-SUS (br.gov.saude.acs, ou o simulador com.guarana.esusmock), mostra uma **pílula flutuante** (TYPE_ACCESSIBILITY_OVERLAY) com o logo e um botão de olho. Fora do e-SUS, a pílula some; o teclado é ignorado.
- O botão abre o Guaraná direto na captura (EXTRA_FROM_ESUS). O **voltar** da captura e do resultado devolve ao e-SUS na mesma tela em que estava (CompanionBridge.backToEsus). O Android 11+ exige <queries> no manifesto para isso.
- **Registrar visita**: o resultado gera os itens da Ficha de Visita Domiciliar ("Busca ativa: Exame", acompanhamentos por condição, desfecho) e um resumo de texto. O preenchimento automático dos campos via árvore de acessibilidade está **em andamento**: o Compose não expõe texto para findAccessibilityNodeInfosByText, e a busca precisa percorrer a árvore.
- **Território**: importado de um JSON exportado (TerritoryStore.import), no formato do simulador. Pessoas viram pacientes com id derivado do CNS.

O integrador oficial de dados é o **LEDI/PEC** (/api/v1/recebimento/ficha, credenciais do PEC, HTTPS). Ele é o caminho para produção; a pílula é o caminho de experiência do agente.

3.10 Exportação e privacidade

Saída	Arquivo	Conteúdo
Relatório PDF	<code>core/Report.kt</code>	A4 com foto, sinais, triagem e explicação
FHIR	<code>core/fhir/FhirExport.kt</code>	Bundle com Patient, Organization (CNES), Encounter e Observations, para a RNDS
ZIP protegido	<code>core/Report.kt</code>	AES-256 (zip4j): PDF + JSON + fotos

Fotos e análises ficam no armazenamento interno do app. Nada sai do aparelho sem ação do agente ou a nuvem ligada nos ajustes.

3.11 Nuvem (`core/RemoteAnalyzer.kt`)

`POST {servidor}/analyze` em multipart (`image`, `child`); a resposta segue o contrato. Se a triagem da nuvem bate com a local, o resultado vira `confirmed`; se não, `divergent`. O cliente está pronto; o servidor ainda não foi montado.

3.12 Validação automática (`core/Validation.kt`)

Em Ajustes → Validação, o agente escolhe uma pasta de fotos (SAF). O app roda **o mesmo caminho** de uma análise real em cada foto, sem salvar no histórico. Se houver `manifest.csv` (`file;labels;triage;framing`), calcula sensibilidade, especificidade e concordância de triagem. O relatório JSON e Markdown vai para `Downloads/guarana-validacao/`. O conjunto de teste é gerado por `treino/gerar_conjunto_validacao.py`.

4. Hardware de luz (`firmware/`)

v0: anel de LEDs RGB WS2812B (ou PWM) num ESP32 ou ESP32-CAM. A câmera é sempre a do tablet; o ESP32 só controla a luz.

Transporte	Firmware	Classe no app
USB serial (padrão, via OTG)	<code>guarana_luz_serial/</code>	<code>UsbSerialLightRig</code>
Bluetooth LE ou Wi-Fi	<code>guarana_luz/</code>	<code>BleLightRig</code> , <code>HttpLightRig</code>
Simulador no Mac	<code>mock/mock_esp.py</code>	<code>HttpLightRig</code>

Protocolo serial (115200 8N1, uma linha por comando, resposta JSON):

```
P          -> {"ok":true,"fw":"guarana-luz-serial 2.0",...}
CFG mode=ws pin=13 n=12 type=GRB flash=4   configura e grava (o app manda ao conectar)
S FFFFFFFF99 todos os LEDs: RR GG BB WW + brilho
0          apaga
```

A **rotina de cores** é texto editável no app: `rótulo; comando; espera ms; foto`, uma linha por passo, com `*` marcando o quadro analisado.

4.1 Ramo de cor (`core/color/Reflectance.kt`)

Com os quadros da rotina, exposição travada e o ambiente subtraído, na máscara de esclera:

Índice	Fórmula	Sentido
icterícia	$\ln(R \text{ no quadro vermelho} / B \text{ no quadro azul})$	bilirrubina absorve ~460 nm
palidez	$\ln(R/G) \text{ no quadro branco}$	menos hemoglobina, menos absorção no verde
hiperemia	$R/(R+G+B) \text{ no quadro branco}$	vermelhidão

A **máscara de esclera** vem do segmentador (fenda menos a íris dilatada, sem pixels estourados), com a heurística de cor como reserva. O app também gera um **red-free** por software (canal verde do quadro branco), que realça os vasos.

Em fotos da internet, sem nenhum controle de luz, os índices já separam icterícia de olho normal (AUROC 0,90) e vermelhidão (0,88). Calibração clínica (mg/dL, g/dL) exige pares foto + laboratório: hoje só NeoJaundice (bilirrubina, pele de RN: Spearman 0,73) e CP-AnemiC (hemoglobina).

5. Modelos

5.1 Inventário

Modelo	Tarefa	Arquitetura	Entrada	No app
ocular_v22.onnx	15 sinais (multirrótulo)	MobileNetV3-Large	320 px	sim (atual)
Perception Encoder S (candidato v2.3)	doenças prioritárias (multiclasse)	ViT-S/16, 24M, Meta	384 px	em treino
enquadramento_v1.onnx	enquadramento (6 classes)	MobileNetV3-Small, cinza	160 px	sim
fenda_v1.onnx	segmentação fenda + íris	MobileNetV3-Small + decodificador	256 px	sim
palidez_v1.onnx	hemoglobina na conjuntiva	MobileNetV3-Small		não (não transfere entre bases)

5.2 Cartão do modelo (`assets/model_card.json`)

Todo modelo publicado vem com um cartão que o app lê em vez de ter valores fixos no código: nome, classes na ordem da saída, tamanho e normalização da entrada, limiares `thresholds` (presente) e `thresholds_candidato`, `unreliable_labels`, contagem de positivos por domínio e métricas de teste.

5.3 Como o encoder transforma a foto em diagnóstico

Para o Perception Encoder S:

```
foto 384x384 → 576 pedaços de 16x16 → Conv2d(3→384): um vetor por pedaço
→ + posição (RoPE) + token CLS → 12 blocos de atenção
→ AttentionPoolLatent: resumo de 384 números
→ Linear(384 → n_doenças) → softmax → probabilidades
→ limiares do cartão + regras da ficha → triagem
```

O pré-treino da Meta ensina o modelo a ver (bordas, cor, estruturas). O ajuste fino com as nossas fotos ensina o que importa no olho. Detalhes e trechos de código: seção 7.

6. Dados

6.1 Fontes

Nenhuma foto foi tirada pelo projeto. Tudo vem de bases públicas:

Tipo	Fontes
Foto comum rotulada (treino)	HF pterígio, HF icterícia, eye_diseases (Kaggle/HF), Faridpur, CFD, conj_seg, slid (lâmpada de fenda)
Fontes novas (metade A treina, metade B testa)	PubMed (figuras), Wikimedia Commons, Mendeley (pterígio/nevo/melanoma, blefarite), pinkeye, Zenodo (1.980 olhos normais de celular), cekmate (leucocoria), gh_eyedis5, Commons por termo
Busca de imagens	Bing + DuckDuckGo, 186 buscas em 4 línguas para 13 condições (<code>treino/minerar_busca.py</code>)
Segmentação	periorbital (CFD + CelebA, máscaras de esclera/íris)
Cor com laboratório	NeoJaundice (bilirrubina), CP-AnemiC (hemoglobina), Eyes-Defy-Anemia (classe)

O catálogo com licenças está no diário técnico. Várias licenças proíbem redistribuição, por isso `dados/` não vai para o git.

6.2 Higiene dos dados

- Desduplicação:** md5 e dHash (distância de Hamming ≤ 6) contra tudo que já existe. Ex.: 1.914 cópias do eye_diseases no pinkeye; 16 no gh_eyedis5; um "dataset novo" de icterícia era cópia byte a byte de outro.
- Portões na entrada:** toda foto nova passa pelo modelo de enquadramento e pelo segmentador (íris e fenda plausíveis). Isso tira rosto inteiro, diagrama, lâmina, animal.
- Metade A / metade B:** cada fonte externa é dividida por hash do sujeito. A metade A pode ir para o treino; a **B é teste para sempre**. Todos os scripts respeitam essa coluna.
- Rótulo ausente não é negativo:** cada fonte anota poucos sinais. Com `OCULAR_LABEL_MASK=1`, a perda só usa os rótulos que a fonte anota (antes, 588 pterígios ensinavam "sem hiperemia").
- Rótulo por texto relido:** legendas do PubMed e descrições do Commons passam pelo LAYA (modelo de decisões tipadas, só texto): achado, tipo de imagem, pós-operatório. Concordância com o rótulo por palavra-chave: 64% no PubMed, 77% no Commons.
- Rótulo de busca verificado:** cada foto da busca é conferida pelo SigLIP 2 (imagem × descrição clínica de cada classe) antes de entrar (`treino/verificar_rotulos.py`), depois de calibrado em rótulos curados.

7. Treino

Ambiente: `.venv` (uv), PyTorch com MPS (GPU do Mac), timm, transformers.

7.1 Scripts

Script	O que treina
<code>treinar.py</code>	modelo de sinais multirrótulo do app (exporta ONNX com saída <code>cam</code> , cartão, INT8)
<code>treinar_doencas.py</code>	doenças prioritárias, multiclasse, com os dois testes (v2.3)
<code>bench_arquiteturas.py</code>	mesma régua para qualquer backbone do timm, EyeCLIP e VisionFM
<code>experimento_300_200.py</code>	4 classes, 300 de treino e 200 de teste por classe
<code>treinar_enquadramento.py</code>	enquadramento, por síntese de janelas sobre íris anotadas
<code>treinar_segmentador.py</code>	fenda e íris (IoU 0,92 / 0,94 na v1)
<code>treinar_palidez.py</code> , <code>calibrar_ictericia.py</code>	ramo de cor com laboratório

`treinar.py` é configurado por variáveis de ambiente:

Variável	Efeito
<code>OCULAR_MODEL</code> , <code>OCULAR_IMG_SIZE</code>	backbone e entrada
<code>OCULAR_AUG=forte</code>	aumentos fortes (cor, borrão, reamostragem, apagamento)
<code>OCULAR_TELA_P=0.3</code>	simula foto de tela (grade de subpixels, moiré, brilho, tom azulado)
<code>OCULAR_BALANCE_SOURCE=1</code>	amostragem balanceada por fonte
<code>OCULAR_LABEL_MASK=1</code>	perda só nos rótulos que a fonte anota
<code>OCULAR_USE_SINTESE=1</code>	inclui dados sintéticos
<code>OCULAR_EXCLUDE_SOURCES=x</code>	deixa uma fonte inteira de fora e a usa como teste
<code>OCULAR_TAG=nome</code>	versão experimental, sem sobrescrever o cartão de produção

7.2 Ajuste fino (o núcleo)

```
m = timm.create_model("vit_pe_core_small_patch16_384.fb", pretrained=True, num_classes=len(CL))

ttr = T.Compose([T.RandomResizedCrop(384, scale=(0.4, 1)), T.RandomHorizontalFlip(),
                 T.RandomRotation(15), T.ColorJitter(0.3, 0.3, 0.15, 0.02), T.ToTensor(),
                 T.Normalize(mean, std)])

lr = 5e-5 # encoder: passos pequenos (preserva o pré-treino)
opt = AdamW([{"params": corpo, "lr": lr}, {"params": cabeca, "lr": lr * 10}], weight_decay=0.05)
sampler = WeightedRandomSampler(1 / contagem_da_classe) # classes raras aparecem tanto quanto as comuns

for x, y in dl: # 12 épocas
    loss = cross_entropy(m(x), y, label_smoothing=0.1)
    loss.backward(); clip_grad_norm_(m.parameters(), 1.0); opt.step(); sched.step()
```

7.3 Síntese (Blender), experimental

`sintese/olho.py` (olho paramétrico) e `sintese/olho_mpf.py` (cabeça MakeHuman CC0 + globo com sinais e máscaras), mais `sintese/embrulhar.py` (projeta foto real na geometria) e `treino/colar_lesoes.py` (cola lesão real em olho normal real). **Resultado:** em cima dos dados reais, não melhorou os testes externos. Fica como ferramenta, fora do treino de produção.

8. Avaliação

8.1 Ferramentas

Script	Mede
<code>testar_externo.py</code>	um modelo numa fonte externa (metade B), com as mesmas vistas do app
<code>comparar_externos.py</code>	várias versões em nove fontes externas → <code>saida/comparacao_externos.md</code>
<code>testar_tela.py</code>	foto de tela simulada (o teste do Google na tela do Mac)
<code>recalibrar_limiares.py</code>	limiares presente/candidato por especificidade em olhos normais reais
<code>bench_conjunto.py</code>	conjuntos de modelos e várias vistas

8.2 Resultados de referência (01/10/2026)

Banco de arquiteturas, 4 condições (normal, conjuntivite, pterígio, catarata), mesmo split, ajuste fino completo. "Google" = fotos da busca nunca vistas, média por classe:

Modelo	Parâmetros	Teste separado	Google	CPU (Mac)
Perception Encoder S	24M	91,2%	92,4%	39 ms
ConvNeXt-T ImageNet-22k	28M	91,1%	92,1%	188 ms
EVA-02 S	22M	90,9%	91,7%	53 ms
SigLIP 2 B/384	93M	90,6%	90,9%	136 ms
VisionFM olho externo	86M	90,5%	89,1%	41 ms
ViT-B DINOv3	86M	91,0%	88,7%	57 ms
MobileNetV3 (app até a v2.2)	4M	85,9%	77,6%	91 ms

Conjunto dos 3 melhores com 6 vistas: 93,1% (+0,7 ponto por 3× o custo). Lição dos experimentos: os saltos vieram de **fontes reais novas** (52% → 77,6%) e de um **backbone moderno** (77,6% → 92,4%); tamanho acima de ~25M, síntese e conjuntos rendem pouco.

8.3 Modos de falha conhecidos

- **Foto de tela:** fotografar a imagem na tela do computador derrubava o pterígio a 0%. Corrigido no treino com a simulação de tela (E5: 0% → 47% presente).
- **Pterígio vermelho e vascular:** lido como uveíte pelo modelo antigo; o Perception Encoder acerta (0,92). Caso de regressão em `validacao/casos/`.
- **Classes com poucos dados:** uveíte, icterícia, hemorragia, ceratite, hanseníase e celulite ainda não têm validação acima de 90%.

9. Publicação no app

```
treino/publicar_modelo.sh ocular_v22
```

1. Marca confiabilidade por domínio no cartão (`marcar_confiabilidade.py`).
2. Avaliação offline com as vistas do app e os casos reais (`avaliar.py --tta`).
3. Gera o conjunto de validação do aparelho.
4. Copia o `.onnx` e o `model_card.json` para `guarana-app/app/src/main/assets/`.
5. Compila o APK (`./gradlew assembleRelease`, JDK 17).

Os `.onnx` não ficam no git; quem clona o repositório precisa treinar ou receber os pesos.

10. Simulador do e-SUS (`esus-mock-app/`)

Réplica das telas do e-SUS Território para testar a integração sem o app oficial: login, unidade, território (bairro > logradouro > imóvel > família), cadastro individual em etapas e a Ficha de Visita Domiciliar com os grupos de motivos ("Busca ativa: Exame", acompanhamentos) e desfechos. Dados fictícios. Pacote `com.guarana.esusmock`, que a pílula reconhece como e-SUS.

11. Comandos

```
# app
cd guarana-app && export JAVA_HOME=/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home
&& ./gradlew assembleRelease
adb install -r app/build/outputs/apk/release/app-release.apk
adb shell settings put secure enabled_accessibility_services
com.guarana.ocular/com.guarana.ocular.esus.EsusCompanionService

# ambiente Python
uv venv .venv && uv pip install --python .venv/bin/python -r treino/requirements.txt

# coleta e preparação
.venv/bin/python treino/minerar_busca.py --por-termo 100 --sinais pterigio,conjuntivite
.venv/bin/python treino/verificar_rotulos.py

# treino e avaliação
.venv/bin/python treino/treinar_doencas.py --modelo vit_pe_core_small_patch16_384.fb
.venv/bin/python treino/bench_arquiteturas.py --modelos mobilenetv3_large_100
vit_pe_core_small_patch16_384.fb
.venv/bin/python treino/comparar_externos.py ocular_v21 ocular_v22

# publicação
treino/publicar_modelo.sh <versao>

# simulador do anel de luz
python3 firmware/mock/mock_esp.py 8080
```

12. Pendências técnicas

Item	Estado
Perception Encoder no app (v2.3), medir latência no tablet	treino das doenças prioritárias em andamento
Preenchimento automático da Ficha de Visita	busca na árvore de acessibilidade a reescrever
Servidor da nuvem (<code>P0ST /analyze</code>)	cliente pronto, servidor a montar
Modo palidez (pálpebra puxada) e modo tracoma	captura dedicada a criar
Calibração clínica do ramo de cor	precisa de pares foto + laboratório
Validação clínica prospectiva	com oftalmologista e ACS em campo
